

AI Assisted Coding

Assignment 6.1

P Sree Varshini

2303A52336

Batch-40

Experiment 6: AI-Based Code Completion: Working with suggestions for classes, loops, conditionals

Task Description #1 (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs.

```
1 #Task-1
2 #Write a python program to print all even numbers from 1 to n using loop.
3 n = int(input("Enter a positive integer n: "))
4 print(f"Even numbers from 1 to {n} are:")
5 for i in range(2, n+1, 2):
6     print(i)
7
8
```

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"
Enter a positive integer n: 52
Even numbers from 1 to 52 are:
2
4
```

```
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50
52
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>
```

Analysis:

The program asks the user to enter a number **n**.

The number is stored in the variable **n**.

The loop starts from **2** because 2 is the first even number.

The loop increases by **2** each time.

Only even numbers are printed up to **n**.

Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

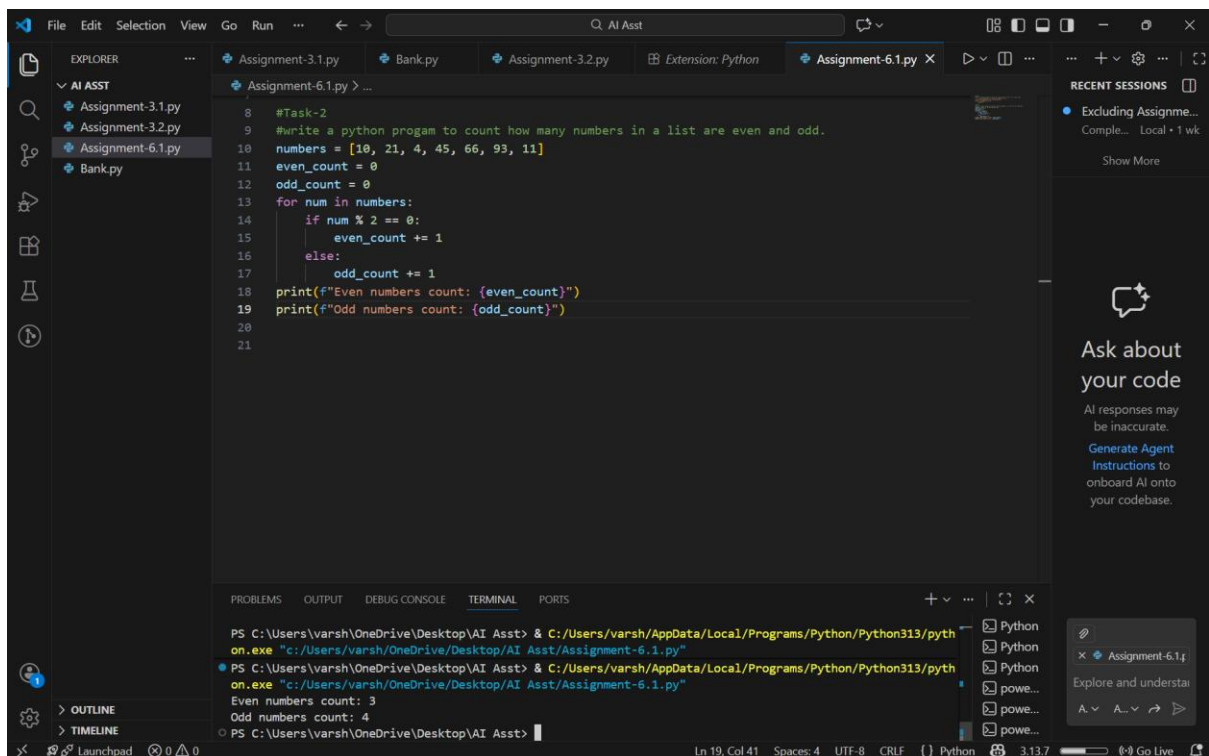
Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow.



```
8 #Task-2
9 #write a python program to count how many numbers in a list are even and odd.
10 numbers = [10, 21, 4, 45, 66, 93, 11]
11 even_count = 0
12 odd_count = 0
13 for num in numbers:
14     if num % 2 == 0:
15         even_count += 1
16     else:
17         odd_count += 1
18 print(f"Even numbers count: {even_count}")
19 print(f"Odd numbers count: {odd_count}")
20
21
```

PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"

PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"

Even numbers count: 3
Odd numbers count: 4

PS C:\Users\varsh\OneDrive\Desktop\AI Asst>

Analysis:

A list of numbers is stored in the variable **numbers**.

Two variables, **even count** and **odd count**, are set to 0.

The program checks each number in the list using a **for loop**.

If a number is divisible by 2, it is **even**, so even count is increased.

Otherwise, the number is **odd**, so odd count is increased.

Finally, the program prints the total count of even and odd numbers.

Task Description #3 (AI-Based Code Completion for Class

Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.

```
21 #Task-3
22 #Write a python class to user that validates age and email using conditional statements.
23 class User:
24     def __init__(self, name, age, email):
25         self.name = name
26         self.age = age
27         self.email = email
28
29     def is_valid_age(self):
30         return 0 < self.age < 120
31
32     def is_valid_email(self):
33         return "@" in self.email and "." in self.email.split("@")[-1]
34
35 # Example usage:
36 user1 = User("Bob", 25, "bob@example.com")
37 print(f"Is valid age: {user1.is_valid_age()}")
38 print(f"Is valid email: {user1.is_valid_email()}")
39 user2 = User("Alice", 130, "aliceexample.com")
40 print(f"Is valid age: {user2.is_valid_age()}")
41 print(f"Is valid email: {user2.is_valid_email()}")
42
```

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/pyth
on.exe "c:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"
Is valid age: True
Is valid email: True
Is valid age: False
Is valid email: False
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>
```

Analysis:

A class named **User** is created.

The init method stores the **name, age, and email** of the user.

The method **is valid age()** checks if the age is between **1 and 119**.

The method **is valid email()** checks if the email:

- contains **@**
- has a **.** after **@**

A user object is created and the validation results are printed.

The same checks are done for another user with invalid details.

Task Description #4 (AI-Based Code Completion for Classes)

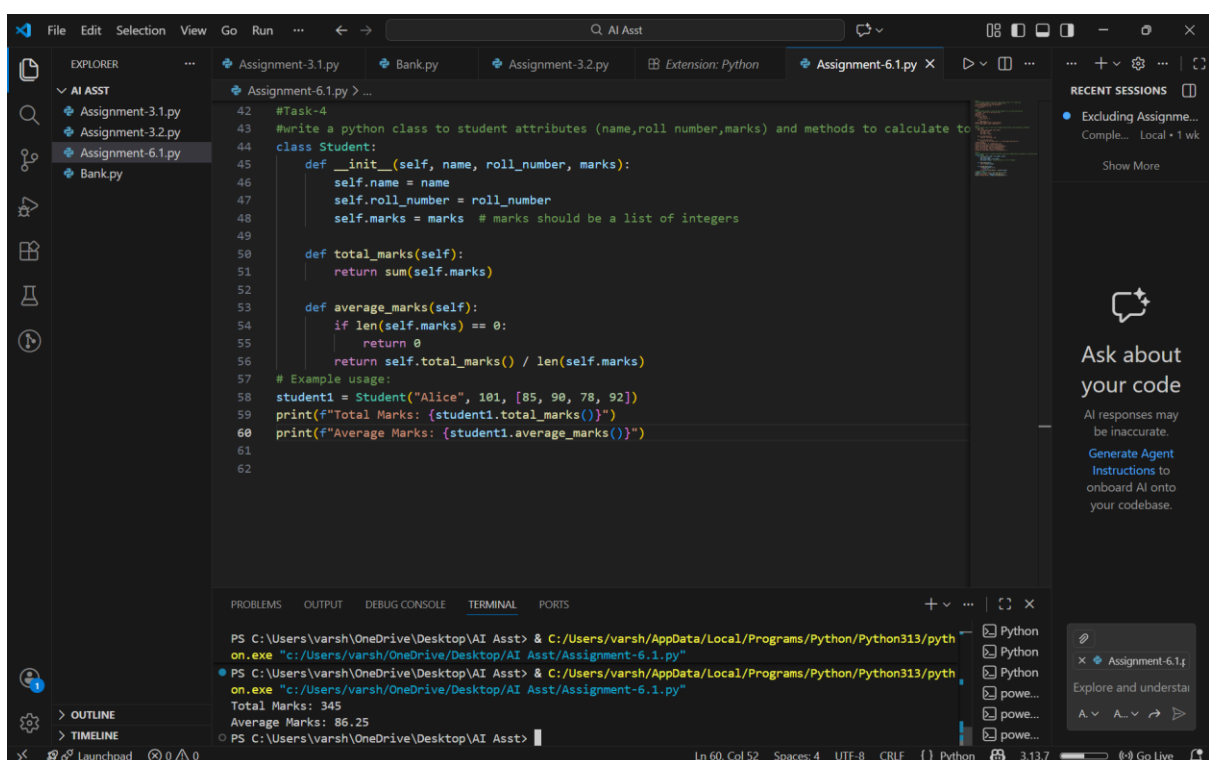
Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements (if needed) with justification.



The screenshot shows a VS Code editor with a Python file named 'Assignment-6.1.py'. The code defines a class 'Student' with attributes 'name', 'roll_number', and 'marks'. It includes methods 'total_marks()' and 'average_marks()'. The 'average_marks()' method checks if the 'marks' list is empty. An example usage is provided, creating a 'Student' object and printing its total and average marks. The terminal at the bottom shows the execution of the script, displaying the output: 'Total Marks: 345' and 'Average Marks: 86.25'.

```
#Task-4
#Write a python class to student attributes (name,roll number,marks) and methods to calculate total and average marks.
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks # marks should be a list of integers

    def total_marks(self):
        return sum(self.marks)

    def average_marks(self):
        if len(self.marks) == 0:
            return 0
        return self.total_marks() / len(self.marks)

# Example usage:
student1 = Student("Alice", 101, [85, 90, 78, 92])
print(f"Total Marks: {student1.total_marks()}")
print(f"Average Marks: {student1.average_marks()}")
```

PS C:\Users\varsh\OneDrive\Desktop\AI Asst> C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"

PS C:\Users\varsh\OneDrive\Desktop\AI Asst> C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"

Total Marks: 345
Average Marks: 86.25

Analysis:

A class named **Student** is created.

The `__init__` method stores the student's **name, roll number, and marks**.

The **total marks()** method adds all the marks and returns the total.

The **average marks()** method:

- checks if the marks list is empty
- calculates the average by dividing total marks by number of subjects

A student object is created and the total and average marks are printed.

Task Description 5 (AI-Assisted Code Completion Review)

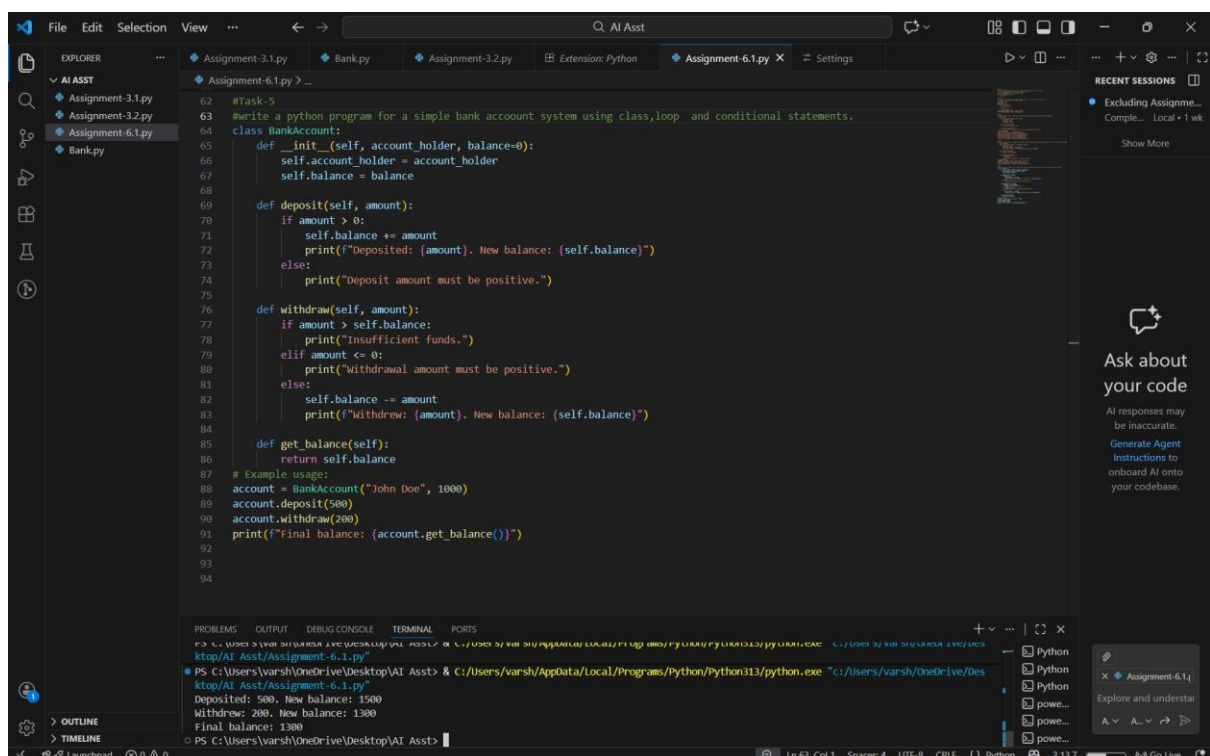
Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity.



```
#Task 5
#Write a python program for a simple bank account system using class,loop and conditional statements.
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}. New balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        elif amount <= 0:
            print("Withdrawal amount must be positive.")
        else:
            self.balance -= amount
            print(f"Withdrew: {amount}. New balance: {self.balance}")

    def get_balance(self):
        return self.balance

# Example usage:
account = BankAccount("John Doe", 1000)
account.deposit(500)
account.withdraw(200)
print(f"Final balance: {account.get_balance()}")
```

Terminal Output:

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/python/python313/python.exe "c:/Users/varsh/OneDrive/Desktop/AI Asst/Assignment-6.1.py"
Deposited: 500. New balance: 1500
Withdrew: 200. New balance: 1300
Final balance: 1300
```

Analysis:

A class named **BankAccount** is created.

The `init` method stores the **account holder name** and **balance**.

The **`deposit()`** method:

- checks if the amount is positive
- adds the amount to the balance

The **`withdraw()`** method:

- checks if there is enough balance
- checks if the amount is positive
- subtracts the amount from the balance

The **`get_balance()`** method returns the current balance.

The program performs deposit, withdrawal, and prints the final balance.